

Trabajo Práctico

Modelado y Optimización

Profesor:

Marcelo Mydlarz

2026/1

Universidad Nacional de General Sarmiento

Integrantes:

Aragón, Evelin

Faccini, Gonzalo

Curcio, Matías

Torres, Pablo

Contents

1	Introducción	3
1.1	Descripción del Problema	3
1.2	Desafíos Operativos	3
1.3	Objetivo General	3
2	Estrategia 1: Salud (Modelo MILP Compacto)	4
2.1	Descripción de la Estrategia	4
2.2	Modelo Matemático	4
2.2.1	Descripción del Problema Específico	4
2.2.2	Conjuntos, Parámetros y Variables de Decisión	4
2.2.3	Formulación Algebraica Completa	5
2.2.4	Explicación de la Función Objetivo y Restricciones	6
3	Estrategia 2: SaludCG (Generación de Columnas)	8
3.1	Descripción de la Estrategia	8
3.2	Modelo Matemático	8
4	Estrategia 3: SaludChallenger (Branch & Price)	9
4.1	Descripción de la Estrategia	9
4.2	Modelo Matemático	10
4.2.1	Problema Maestro (formulación por rutas)	10
4.2.2	Subproblema de Pricing	11
4.2.3	Reglas de Ramificación	12
4.3	Pseudocódigo y Detalles de Implementación	13
4.3.1	Detalles de implementación	16
5	Evaluación Comparativa	18
5.1	Configuración de Evaluación	18
5.2	Tabla Comparativa	18
5.3	Abreviaciones	18
6	Conclusiones	20
6.1	Hallazgos Principales	20
6.2	Comportamientos Sorprendentes	20
6.3	Dificultades y Conocimientos Adquiridos	20

1 Introducción

El trabajo práctico se enfoca en resolver un problema fundamental de logística de salud integral: optimizar el traslado de pacientes desde sus domicilios hacia un centro médico de alta complejidad, en contextos donde los recursos (vehículos) son limitados y las restricciones operativas son complejas.

1.1 Descripción del Problema

Una empresa de logística médica opera una flota heterogénea de vehículos (combis) basadas en un centro médico central. Cada día, debe atender solicitudes de traslado de pacientes desde sus hogares al centro. Debido a la naturaleza médica de los traslados y a limitaciones de recursos, la empresa no está obligada a recoger a todos los pacientes, sino que debe seleccionar estratégicamente cuáles atender para maximizar el beneficio social y operativo del sistema.

Cada paciente presenta características específicas:

- **Ubicación geográfica:** coordenadas (x_i, y_i) en el plano
- **Ventana horaria:** intervalo $[ih_{\text{inicio}}, ih_{\text{fin}}]$ dentro del cual debe ser recogido
- **Categoría médica:** clasificación según protocolo médico (p.ej., inmunodeprimido, infeccioso, movilidad reducida)
- **Beneficio:** valor social/económico de atender al paciente

La flota de vehículos también es heterogénea:

- **Capacidad de asientos:** cantidad máxima de pacientes por vehículo
- **Costo operativo:** costo de poner en servicio el vehículo por día
- **Disponibilidad:** cantidad de unidades de cada tipo

1.2 Desafíos Operativos

- **Incompatibilidades médicas:** Ciertas categorías de pacientes no pueden compartir vehículo por razones de bioseguridad o protocolo médico.
- **Ventanas de tiempo:** Cada paciente tiene una ventana horaria estricta. No se permite recoger antes de ih_{inicio} ni después de ih_{fin} .
- **Selección estratégica:** No todos los pacientes pueden ser atendidos; debe maximizarse el beneficio neto (beneficio de pacientes atendidos menos costo operativo de vehículos).
- **Optimización de rutas:** Diseñar rutas eficientes que respeten capacidad, tiempos y compatibilidades.
- **Heterogeneidad de recursos:** Diferentes tipos de vehículos con distintas características implican complejidad combinatoria.

1.3 Objetivo General

Desarrollar e implementar tres estrategias de resolución con complejidad creciente:

1. **Salud (Modelo MILP Compacto):** Formulación algebraica directa utilizando programación lineal entera mixta.
2. **SaludCG (Generación de Columnas):** Descomposición del problema mediante generación de columnas para mejorar escalabilidad.
3. **SaludChallenger (Branch & Price):** Mejoras avanzadas basadas en la literatura (descomposición, eliminación de columnas, inicialización eficiente).

Todas las estrategias deben incluir un módulo de validación (**SaludTest**) que verifica factibilidad sin usar programación lineal.

2 Estrategia 1: Salud (Modelo MILP Compacto)

2.1 Descripción de la Estrategia

La estrategia Salud resuelve el problema mediante un modelo de **Programación Lineal Entera Mixta (MILP) compacto**. Este enfoque formula el problema completo en un único modelo algebraico que incluye todas las restricciones operativas, y lo resuelve directamente utilizando un solver de optimización (en este caso, PySCIPOpt sobre el solver SCIP).

Ventajas:

- Garantiza optimalidad (o cotas de optimalidad) dentro del tiempo permitido
- Implementación directa y comprensible
- Adecuada para instancias pequeñas a medianas

Limitaciones:

- Escalabilidad limitada para instancias grandes
- Tiempo de resolución puede crecer exponencialmente con el tamaño
- Espacio de soluciones potencialmente muy grande

2.2 Modelo Matemático

2.2.1 Descripción del Problema Específico

Se busca diseñar un conjunto de rutas (una por cada vehículo) que:

- Seleccionen qué pacientes serán atendidos
- Asignen cada paciente atendido a exactamente un vehículo
- Definan el orden de visita para cada vehículo
- Respeten todas las restricciones operativas (capacidad, tiempo, incompatibilidades)
- Maximicen el beneficio neto total

2.2.2 Conjuntos, Parámetros y Variables de Decisión

Conjuntos

- P : Conjunto de pacientes solicitantes, $P = \{1, 2, \dots, m\}$
- $V = \{0\} \cup P$: Conjunto de nodos (centro médico + pacientes)
- K : Conjunto de combis individuales disponibles
- T : Conjunto de tipos de combis (Combi_Chica, Combi_Mediana, etc.)
- $\mathcal{C}$: Conjunto de categorías médicas
- $\mathcal{I} \subseteq \mathcal{C} \times \mathcal{C}$: Conjunto de pares de categorías incompatibles

Parámetros

- $\text{beneficio}[p]$: Beneficio de atender al paciente p (valor numérico)
- $\text{costo}[t]$: Costo operativo del tipo de combi t
- $\text{capacidad}[k]$: Número máximo de asientos en la combi k
- (x_p, y_p) : Coordenadas geográficas del paciente p
- (x_0, y_0) : Coordenadas del centro médico (nodo 0)
- $d[i, j] = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$: Distancia euclídea entre nodos i y j
- $[ih_{\text{inicio}}[p], ih_{\text{fin}}[p]]$: Ventana de tiempo para el paciente p
- $\text{categoria}[p]$: Categoría médica del paciente p
- $\text{tipo}[k]$: Tipo de combi al que pertenece la combi k
- $M = 10000$: Constante Big-M para linealización de restricciones

Variables de Decisión

- $x[i, j, k] \in \{0, 1\}$: Binaria, igual a 1 si la combi k viaja directamente del nodo i al nodo j
- $z[p, k] \in \{0, 1\}$: Binaria, igual a 1 si el paciente p es atendido por la combi k
- $a[p] \in \{0, 1\}$: Binaria, igual a 1 si el paciente p es atendido (por alguna combi)
- $u[k] \in \{0, 1\}$: Binaria, igual a 1 si la combi k es utilizada (realiza algún viaje)
- $T[i, k] \geq 0$: Continua, tiempo de llegada (o inicio de servicio) a nodo i con combi k

2.2.3 Formulación Algebraica Completa

$$\max \quad Z = \sum_{p \in P} \text{beneficio}[p] \cdot a[p] - \sum_{k \in K} \text{costo}[\text{tipo}[k]] \cdot u[k] \quad (1)$$

Restricciones

1. Cobertura de Pacientes

$$\sum_{k \in K} z[p, k] \leq 1 \quad \forall p \in P \quad (2)$$

$$a[p] = \sum_{k \in K} z[p, k] \quad \forall p \in P \quad (3)$$

Ecuación (2): Cada paciente es atendido por a lo sumo una combi.

Ecuación (3): Un paciente se considera atendido si está asignado a alguna combi.

2. Flujo de Rutas

$$\sum_{j \in V, j \neq i} x[i, j, k] = \sum_{j \in V, j \neq i} x[j, i, k] \quad \forall i \in V, k \in K \quad (4)$$

Conservación de flujo: Todo lo que entra a un nodo debe salir (excepto si no se visita).

3. Operación del Centro Médico

$$\sum_{j \in P} x[0, j, k] = u[k] \quad \forall k \in K \quad (5)$$

$$\sum_{i \in P} x[i, 0, k] = u[k] \quad \forall k \in K \quad (6)$$

Ecuación (5): Si la combi k es utilizada, debe partir desde el centro.

Ecuación (6): Si la combi k es utilizada, debe regresar al centro.

4. Restricción de Capacidad

$$\sum_{p \in P} z[p, k] \leq \text{capacidad}[k] \cdot u[k] \quad \forall k \in K \quad (7)$$

El número de pacientes en la combi k no puede exceder su capacidad.

5. Restricciones de Ventana de Tiempo

$$T[0, k] = 0 \quad \forall k \in K \quad (8)$$

$$T[i, k] \geq T[j, k] + d[j, i] - M(1 - x[j, i, k]) \quad \forall i, j \in V, k \in K, i \neq j \quad (9)$$

$$T[p, k] \geq ih_{\text{inicio}}[p] - M(1 - z[p, k]) \quad \forall p \in P, k \in K \quad (10)$$

$$T[p, k] \leq ih_{\text{fin}}[p] + M(1 - z[p, k]) \quad \forall p \in P, k \in K \quad (11)$$

Ecuación (8): El tiempo en el centro al inicio es 0.

Ecuación (9): Propagación del tiempo: si se viaja de j a i con combi k , el tiempo en i es al menos el tiempo en j más la distancia.

Ecuaciones (10) y (11): Si el paciente p es atendido por combi k , debe ser recogido dentro de su ventana de tiempo.

6. Restricciones de Incompatibilidad

$$z[p, k] + z[q, k] \leq 1 \quad \forall k \in K, (p, q) \in \mathcal{I}_{\text{pacientes}} \quad (12)$$

donde $\mathcal{I}_{\text{pacientes}} = \{(p, q) : \text{categoria}[p], \text{categoria}[q] \in \mathcal{I}\}$

Si dos pacientes pertenecen a categorías incompatibles, no pueden estar en el mismo vehículo.

7. Vínculo entre llegar al vértice y atender al paciente

$$z[p, k] \leq \sum_{i \in P, i \neq p} x[i, p, k] \quad \forall k \in K, p \in P \quad (13)$$

Si un paciente p es atendido por la combi k , dicha combi debe haber pasado por el vértice de p al menos una vez.

2.2.4 Explicación de la Función Objetivo y Restricciones

Función Objetivo (Ec. (1)) Maximizar el **beneficio neto** total:

$$Z = \sum_{p \in P} \text{beneficio}[p] \cdot a[p] - \sum_{k \in K} \text{costo}[\text{tipo}[k]] \cdot u[k]$$

Componentes:

- **Primer término:** Suma de beneficios de todos los pacientes atendidos.
- **Segundo término:** Costo total de operación de los vehículos utilizados (se resta porque es un costo).

El modelo busca el equilibrio entre maximizar beneficios y minimizar costos operativos.

Restricciones de Cobertura (Ecs. (2), (3)) Aseguran que:

- Cada paciente puede ser atendido por **a lo sumo una combi** (no hay duplicación)
- La variable $a[p]$ captura correctamente si el paciente fue atendido

Restricción de Flujo (Ec. (4)) Asegura que cada vehículo forma una **ruta conexa**:

- Todo lo que "entra" a un nodo debe "salir"
- Previene ciclos desconectados (subtours)

Restricciones de Centro Médico (Ecs. (5), (6)) Garantizan que:

- Todo vehículo utilizado **parte del centro** al inicio
- Todo vehículo utilizado **regresa al centro** al final

Restricción de Capacidad (Ec. (7)) Respetar la **capacidad máxima** de cada vehículo:

- Número de pacientes $\leq$ capacidad del vehículo

Restricciones de Ventana de Tiempo (Ecs. (8)–(11)) Garantizan cumplimiento de horarios:

- El tiempo en el centro al inicio es cero (tiempo de partida)
- El tiempo se propaga correctamente a lo largo de la ruta
- Si un paciente es atendido, debe ser recogido dentro de su ventana $[ih_{\text{inicio}}, ih_{\text{fin}}]$

Restricciones de Incompatibilidad (Ec. (12)) Aseguran que:

- Dos pacientes con categorías médicas incompatibles **no pueden compartir vehículo**
- Previene conflictos por bioseguridad o protocolo médico

Vínculo entre llegar al vértice y atender al paciente (Ec. (13)) Asegura que:

- Si un paciente fue atendido, entonces al menos una combi pasó por su ubicación.
- Si ninguna combi pasó por su ubicación, entonces no fue atendido.

3 Estrategia 2: SaludCG (Generación de Columnas)

3.1 Descripción de la Estrategia

[SECCIÓN A COMPLETAR]

La estrategia SaludCG implementa un enfoque de **Generación de Columnas (Column Generation, CG)** para resolver el problema de manera más escalable. Este método descompone el problema original en:

- **Problema Maestro:** Selecciona rutas de un conjunto disponible, minimizando costo/maximizando beneficio
- **Subproblema de Pricing:** Genera nuevas rutas con costo reducido negativo

El proceso itera hasta convergencia LP, permitiendo manejar instancias más grandes.

3.2 Modelo Matemático

[A COMPLETAR]

4 Estrategia 3: SaludChallenger (Branch & Price)

4.1 Descripción de la Estrategia

La estrategia **SaludChallenger** convierte la generación de columnas de la Estrategia 2 en un método *exacto* mediante **Branch & Price (B&P)**, la técnica estándar de la literatura para problemas de ruteo resueltos por generación de columnas. La motivación surge de dos limitaciones de SaludCG:

- **Su etapa final es heurística.** Cuando SaludCG deja de generar columnas, resuelve el maestro *entero* usando únicamente las rutas de su pool. Nada garantiza que la solución entera óptima se forme con esas rutas: puede necesitar columnas que la fase LP nunca tuvo motivo de generar.
- **No tiene mecanismo para cerrar el gap.** La cota dual de SaludCG corresponde a la relajación lineal del nodo raíz; si esa relajación es fraccionaria, el método no dispone de ninguna herramienta para acercar la cota al valor de la mejor solución entera.

Branch & Price ataca ambos problemas integrando la generación de columnas dentro de un árbol de *branch & bound*: la relajación lineal de **cada nodo** del árbol se resuelve por generación de columnas (con lo cual la cota de cada nodo es válida aunque el maestro tenga pocas columnas), y cuando la solución de un nodo es fraccionaria se **ramifica** usando reglas elegidas especialmente para que las decisiones puedan imponerse también dentro del subproblema de pricing. Con tiempo suficiente, el método encuentra el óptimo entero y además *certifica* su optimalidad.

Vista general del método

El algoritmo se organiza en seis pasos; los pseudocódigos correspondientes están en la Sección 4.3.

1. **Inicialización del pool de columnas** (Algoritmo 3): se generan todas las rutas *singleton* factibles (ida y vuelta a cada paciente alcanzable, por cada tipo de combi) más rutas multi-paciente construidas por una heurística golosa que respeta ventanas de tiempo, capacidad e incompatibilidades.
2. **Warm-start del incumbente:** antes de explorar el árbol se resuelve el maestro entero sobre el pool inicial. Su valor (o $Z = 0$, “no operar”, que siempre es factible) es el primer incumbente y habilita podas desde el primer nodo.
3. **Exploración best-first:** los nodos abiertos se guardan en una cola de prioridad ordenada por la cota LP de su padre; siempre se procesa primero el nodo más prometedor.
4. **Generación de columnas en el nodo** (Algoritmo 2): se resuelve la relajación lineal del maestro restringido a las columnas compatibles con las decisiones del nodo, iterando maestro $\leftrightarrow$ pricing hasta que ningún tipo de combi aporte columnas de ganancia reducida positiva. Durante estas iteraciones se *eliminan* las columnas que llevan K resoluciones consecutivas sin usarse.
5. **Poda o ramificación:** si la cota LP del nodo no supera al incumbente, el nodo se descarta; si la solución es entera, se actualiza el incumbente; si es fraccionaria, se crean dos hijos con la primera regla aplicable (Ryan–Foster sobre pares de pacientes, atención de un paciente, o cantidad de vehículos por tipo).
6. **Refuerzo final:** al agotarse el árbol o el tiempo, se resuelve una última vez el maestro entero sobre *todo* el pool acumulado, que puede mejorar el incumbente combinando rutas generadas en distintos nodos.

Relación con las mejoras sugeridas en el enunciado

- *Generar las columnas iniciales de manera más eficiente:* singletons factibles más heurística golosa (Algoritmo 3); el pool inicial ya contiene rutas de buena calidad, y el warm-start las convierte de inmediato en una solución factible.

- *Eliminar las columnas no usadas en las últimas iteraciones:* contador *sin_uso* por columna con umbral $K = 25$. Las columnas de la mejor solución conocida están protegidas y, si el pricing vuelve a generar una columna eliminada, ésta se reactiva; así la limpieza no invalida las cotas (detalles en la Sección 4.3.1).
- *Mejoras basadas en la literatura:* el esquema completo de Branch & Price con ramificación de Ryan–Foster.

Ventajas:

- Método exacto: si el árbol se agota dentro del umbral, la solución queda *certificada* como óptima (a diferencia de la etapa final heurística de SaludCG)
- Cotas duales válidas en todo momento, lo que permite reportar el gap de optimalidad al detenerse por tiempo
- El maestro de cada nodo trabaja con pocas variables (solo las rutas generadas y compatibles con el nodo)

Limitaciones:

- El subproblema de pricing sigue siendo un MILP de ruteo (NP-difícil); es el cuello de botella en instancias grandes
- El árbol puede crecer en instancias con mucha simetría (columnas intercambiables entre combis idénticas)

4.2 Modelo Matemático

La estrategia utiliza dos modelos que dialogan entre sí: el **problema maestro**, que selecciona rutas, y el **subproblema de pricing**, que construye rutas nuevas. Se describen a continuación, junto con las reglas de ramificación que los conectan con el árbol de búsqueda.

4.2.1 Problema Maestro (formulación por rutas)

i. Problema que resuelve. Decidir *qué rutas ejecutar* para maximizar el beneficio neto, suponiendo conocido el conjunto de todas las rutas factibles. Una *ruta* es el recorrido de una combi que parte del centro, recoge un subconjunto de pacientes en cierto orden y regresa al centro, respetando capacidad, ventanas de tiempo e incompatibilidades. Esta formulación traslada toda la complejidad del ruteo a la definición de las columnas: al maestro solo le queda combinarlas.

ii. Conjuntos, parámetros y variables.

- P : pacientes; T : tipos de combi
- Ω_t : conjunto (de tamaño exponencial) de rutas factibles para el tipo t ; $\Omega = \bigcup_{t \in T} \Omega_t$
- $a_{pr} \in \{0, 1\}$: parámetro igual a 1 si la ruta r atiende al paciente p
- $c_r = \sum_{p \in P} a_{pr} \cdot \text{beneficio}[p] - \text{costo}[t(r)]$: rentabilidad neta de la ruta r (el beneficio de sus pacientes menos el costo operativo de su combi)
- $\text{disp}[t]$: cantidad de combis disponibles del tipo t
- Variable de decisión: $y_r \in \{0, 1\}$, igual a 1 si la ruta r se ejecuta

iii. Formulaci3n.

$$\max \quad Z = \sum_{r \in \Omega} c_r \cdot y_r \quad (14)$$

$$\text{s.a.} \quad \sum_{r \in \Omega} a_{pr} \cdot y_r \leq 1 \quad \forall p \in P \quad (15)$$

$$\sum_{r \in \Omega_t} y_r \leq \text{disp}[t] \quad \forall t \in T \quad (16)$$

$$y_r \in \{0, 1\} \quad \forall r \in \Omega \quad (17)$$

iv. Explicaci3n. El objetivo (14) suma la rentabilidad neta de las rutas elegidas; coincide con el beneficio neto del enunciado porque cada ruta ya descuenta el costo de su combi. La restricci3n (15) impide que un paciente sea atendido por m1s de una ruta (atenderlo o no queda impl3cito: la desigualdad permite dejarlo sin cubrir), y (16) que se usen m1s veh3culos de un tipo que los disponibles. Como $|\Omega|$ es exponencial, el modelo completo nunca se materializa: en cada nodo del 1rbol se resuelve la *relajaci3n lineal* ($y_r \geq 0$) sobre el subconjunto de columnas generado hasta el momento —el *maestro restringido*— y el pricing agrega las que falten.

El maestro restringido dentro de un nodo. En un nodo del 1rbol, el LP se ajusta a las decisiones de ramificaci3n vigentes:

- Participan solo las columnas del pool *compatibles* con el nodo (por ejemplo, en una rama “ p y q separados” se excluyen las rutas que los contienen a ambos).
- Si la ramificaci3n exige atender al paciente p , la restricci3n (15) pasa a igualdad. Para que el LP nunca se vuelva infactible (y sus duales queden bien definidos) se agrega una variable artificial $s_p \in [0, 1]$ penalizada con un coeficiente ρ en el objetivo: $\sum_r a_{pr} y_r + s_p = 1$. Con ρ mayor que cualquier beneficio alcanzable, $s_p > 0$ solo ocurre si las decisiones del nodo son incompatibles entre s3; en ese caso el nodo se poda.
- Las ramificaciones sobre la flota reemplazan (16) por cotas $lb_t \leq \sum_{r \in \Omega_t} y_r \leq ub_t$ (la cota inferior tambi3n lleva su variable artificial penalizada).

4.2.2 Subproblema de Pricing

i. Problema que resuelve. Dados los precios sombra del maestro restringido, encontrar —para cada tipo de combi t — la ruta factible de *m1xima ganancia reducida*: la columna que m1s mejoraría el LP actual si se agregara. Es un problema de ruteo de un solo veh3culo con selecci3n de pacientes.

Sean π_p los duales de (15) y μ_t los de (16) (o de sus cotas en el nodo). El costo reducido de una ruta $r \in \Omega_t$ es

$$\bar{c}_r = c_r - \sum_{p \in P} a_{pr} \pi_p - \mu_t = \sum_{p \in P} a_{pr} (\text{beneficio}[p] - \pi_p) - \text{costo}[t] - \mu_t,$$

por lo que basta maximizar la suma $\sum_p (\text{beneficio}[p] - \pi_p) z_p$ sobre las rutas factibles y restar al final las constantes $\text{costo}[t] + \mu_t$.

ii. Conjuntos, par1metros y variables. Para un tipo t fijo, con $V = \{0\} \cup P$ los nodos, $n = |P|$, d_{ij} las distancias, $\text{asientos}[t]$ la capacidad y M una constante grande:

- $x_{ij} \in \{0, 1\}$: la combi viaja directamente del nodo i al nodo j
- $z_p \in \{0, 1\}$: la ruta atiende al paciente p
- $T_i \geq 0$: tiempo de llegada al nodo i
- $u_p \in [0, n]$: posici3n del paciente p en el recorrido (orden tipo MTZ)

iii. Formulaci3n.

$$\max \sum_{p \in P} (\text{beneficio}[p] - \pi_p) \cdot z_p \quad (18)$$

$$\text{s.a.} \sum_{p \in P} z_p \leq \text{asientos}[t] \quad (19)$$

$$\sum_{i \in V, i \neq p} x_{ip} = z_p, \quad \sum_{j \in V, j \neq p} x_{pj} = z_p \quad \forall p \in P \quad (20)$$

$$\sum_{j \in P} x_{0j} \leq 1, \quad \sum_{i \in P} x_{i0} = \sum_{j \in P} x_{0j} \quad (21)$$

$$z_p \leq \sum_{j \in P} x_{0j} \quad \forall p \in P \quad (22)$$

$$T_j \geq T_i + d_{ij} - M \cdot (1 - x_{ij}) \quad \forall i \in V, j \in P, i \neq j \quad (23)$$

$$u_j \geq u_i + 1 - n \cdot (1 - x_{ij}) \quad \forall i, j \in P, i \neq j \quad (24)$$

$$ih_{\text{inicio}}[p] \cdot z_p \leq T_p \leq ih_{\text{fin}}[p] \cdot z_p + M \cdot (1 - z_p) \quad \forall p \in P \quad (25)$$

$$z_p + z_q \leq 1 \quad \forall (p, q) : (\text{categoria}[p], \text{categoria}[q]) \in \mathcal{I} \quad (26)$$

y, seg3n las decisiones del nodo (con J los pares “juntos”, S los pares “separados” y F los pacientes prohibidos):

$$z_a = z_b \quad \forall (a, b) \in J, \quad z_a + z_b \leq 1 \quad \forall (a, b) \in S, \quad z_p = 0 \quad \forall p \in F \quad (27)$$

iv. Explicaci3n. El objetivo (18) maximiza la parte variable de la ganancia reducida: cada paciente “vale” su beneficio menos el precio sombra π_p que el maestro ya paga por cubrirlo. (19) limita los asientos. (20) vincula flujo y atenci3n: la combi entra y sale exactamente una vez de cada paciente atendido y no pasa por los no atendidos. (21) permite a lo sumo un viaje, que si existe sale del centro y regresa a 3l; (22) impide “rutas fantasma” (atender pacientes sin salir del centro). (23) propaga el tiempo a lo largo de los arcos usados; al ser una desigualdad, la combi puede *esperar* en la puerta hasta ih_{inicio} , tal como exige el enunciado. (25) obliga a recoger a cada paciente atendido dentro de su ventana. (24) es un orden tipo Miller–Tucker–Zemlin entre pacientes: elimina los subtours que (23) no alcanza a prohibir cuando hay pacientes a distancia 0 (dos domicilios en la misma coordenada permitir3an un ciclo de duraci3n nula). (26) replica las incompatibilidades de bioseguridad y (27) impone las decisiones de ramificaci3n del nodo. Los pacientes *requeridos* por la ramificaci3n no aparecen aqu3: exigir cobertura es una condici3n global sobre el conjunto de rutas y se impone en el maestro.

Criterio de incorporaci3n. Si θ_t^* es el 3ptimo de (18), la mejor ruta del tipo t tiene ganancia reducida $\theta_t^* - \text{costo}[t] - \mu_t$. Si supera la tolerancia $\varepsilon = 10^{-4}$, la ruta se agrega al pool. Cuando ning3n tipo aporta columnas, el LP del nodo es 3ptimo y su valor es una cota dual v3lida para todo el sub3rbol.

4.2.3 Reglas de Ramificaci3n

Por qu3 no se ramifica sobre y_r . La ramificaci3n cl3sica sobre una variable ($y_r = 0$ vs. $y_r = 1$) es incompatible con la generaci3n de columnas: en la rama $y_r = 0$ el pricing tender3a a regenerar exactamente la ruta prohibida (sigue siendo la m3s atractiva), y excluir *una ruta puntual* no puede expresarse como restricci3n lineal del pricing sin romper su estructura. Adem3s el 3rbol queda desbalanceado: $y_r = 1$ fija mucho y $y_r = 0$ casi nada. Por eso se ramifica sobre cantidades agregadas de la soluci3n, que s3 se traducen en restricciones lineales simples del pricing y en filtros sobre el pool.

Si la soluci3n LP del nodo es fraccionaria, se aplica la primera regla utilizable:

1. **Ryan–Foster (pares de pacientes):** se calcula $w_{pq} = \sum_{r: p, q \in r} y_r$ (cuánto “viajan juntos” p y q) y se elige el par de valor más fraccionario (más cercano a 0,5). En la rama *juntos* se exige $z_p = z_q$ en el pricing y se descartan del maestro las columnas que contienen exactamente uno de los dos; en la rama *separados* se exige $z_p + z_q \leq 1$ y se descartan las columnas que los contienen a ambos. Ambas ramas recortan el espacio de forma equilibrada.
2. **Atención de un paciente:** si la cobertura $\alpha_p = \sum_r a_{pr} \cdot y_r$ es fraccionaria, en una rama se fuerza la atención de p (la restricción (15) pasa a igualdad, con su variable artificial penalizada) y en la otra se prohíbe ($z_p = 0$ en el pricing y filtrado de las columnas que lo contienen).
3. **Vehículos por tipo:** si la cantidad de vehículos usados de un tipo, $v_t = \sum_{r \in \Omega_t} y_r$, es fraccionaria, se ramifica en $v_t \leq \lfloor v_t \rfloor$ y $v_t \geq \lceil v_t \rceil$.

Queda un caso borde: columnas *equivalentes* (mismo tipo de combi y mismo conjunto de pacientes, en distinto orden) pueden repartirse valor fraccionario sin que ninguna regla aplique, porque todas las cantidades agregadas resultan enteras. Como esas columnas tienen idéntica rentabilidad, basta elegir un representante por grupo: se obtiene una solución entera del mismo valor que el LP y el nodo se cierra.

4.3 Pseudocódigo y Detalles de Implementación

Los Algoritmos 1–3 presentan el método en tres niveles: el esquema principal de Branch & Price, la generación de columnas dentro de un nodo (que incluye la eliminación de columnas) y la construcción del pool inicial.

Algoritmo 1 SALUDCHALLENGER — esquema principal de Branch & Price

Entrada: instancia (pacientes P , centro, flota, incompatibilidades), umbral de tiempo

Salida: archivo de salida con la mejor solución entera hallada

```
1: leer la instancia y calcular la matriz de distancias;  $fin \leftarrow \text{ahora} + \text{umbral}$ 
2:  $pool \leftarrow \text{COLUMNASINICIALES}()$  ▷ Algoritmo 3
3:  $(Z^*, R^*) \leftarrow \text{MAESTROENTERO}(pool)$  ▷ warm-start; “no operar” garantiza  $Z^* \geq 0$ 
4:  $abiertos \leftarrow \{\text{nodo raíz, sin decisiones de ramificación}\}$ 
5: mientras  $abiertos \neq \emptyset$  y resta tiempo (descontada la reserva final) y nodos < MAX hacer
6:    $n \leftarrow \text{nodo de mejor cota heredada de } abiertos$  ▷ best-first
7:   si  $\text{cota}(n) \leq Z^*$  entonces continuar ▷ poda por cota heredada
8:   fin si
9:    $(obj, y, artif, \text{convergió}) \leftarrow \text{CGENNODO}(n, pool, fin)$  ▷ Algoritmo 2
10:  si no convergió entonces salir del ciclo ▷ se agotó el tiempo
11:  fin si
12:  si LP infactible o  $artif$  o  $obj \leq Z^*$  entonces continuar ▷ poda
13:  fin si
14:  si  $y$  es entera entonces actualizar  $(Z^*, R^*)$  y continuar
15:  fin si
16:   $d \leftarrow \text{primera regla aplicable: Ryan-Foster} \rightarrow \text{atención de paciente} \rightarrow \text{flota por tipo}$ 
17:  si no hay regla aplicable entonces ▷ columnas equivalentes
18:    redondear por grupos; actualizar  $(Z^*, R^*)$  si mejora; continuar
19:  fin si
20:  crear los dos hijos de  $n$  según  $d$ , con cota heredada  $obj$ , y agregarlos a  $abiertos$ 
21: fin mientras
22: si  $abiertos = \emptyset$  y no se cortó por tiempo ni por nodos entonces marcar óptimo certificado
23: fin si
24:  $(Z_{ip}, R_{ip}) \leftarrow \text{MAESTROENTERO}(pool)$  con el tiempo restante ▷ refuerzo final
25: si  $Z_{ip} > Z^*$  entonces  $(Z^*, R^*) \leftarrow (Z_{ip}, R_{ip})$ 
26: fin si
27: escribir  $Z^*$ , el itinerario de cada ruta de  $R^*$  y los pacientes no atendidos
```

Algoritmo 2 CGENNODO — generación de columnas en un nodo, con eliminación de columnas

Entrada: nodo n , $pool$ global de columnas, fin (instante límite)

Salida: cota LP del nodo, solución y , uso de artificiales, indicador de convergencia

```
1: construir el pricing de cada tipo  $t$  con las restricciones (27) de  $n$  ▷ se reutiliza en todas las iteraciones
2: repetir
3:   si se alcanzó  $fin$  entonces devolver el último LP resuelto, con  $convergió = falso$ 
4:   fin si
5:    $activas \leftarrow \{r \in pool : r \text{ no eliminada y compatible con } n\}$ 
6:   resolver el LP maestro restringido sobre  $activas \Rightarrow obj, y$ , duales  $(\pi, \mu)$ 
7:   para cada  $r \in activas$  hacer ▷ eliminación de columnas sin uso
8:     si  $y_r > 0$  entonces  $sin\_uso_r \leftarrow 0$ 
9:     si no  $sin\_uso_r \leftarrow sin\_uso_r + 1$ 
10:    fin si
11:    si  $sin\_uso_r \geq K$  y  $r$  no pertenece al incumbente entonces marcar  $r$  como eliminada
12:    fin si
13:  fin para
14:   $nuevas \leftarrow 0$ 
15:  para cada tipo de combi  $t$  hacer
16:    resolver el pricing de  $t$  con  $(\pi, \mu_t) \Rightarrow$  mejor ruta  $r^*$  y ganancia reducida  $\bar{c}^*$ 
17:    si  $\bar{c}^* > \varepsilon$  entonces agregar  $r^*$  al pool (si ya existía eliminada, revivirla);  $nuevas \leftarrow nuevas + 1$ 
18:    fin si
19:  fin para
20:  si  $nuevas = 0$  entonces devolver ( $obj, y, artif$ , verdadero) ▷ el LP del nodo es óptimo
21:  fin si
22: fin repetir
```

Algoritmo 3 COLUMNASINICIALES — singletons factibles + heurística golosa

Entrada: pacientes P , centro, flota, distancias d , incompatibilidades

Salida: pool inicial de columnas

```
1:  $p$  es alcanzable  $\iff \max(d_{0p}, ih_{\text{inicio}}[p]) \leq ih_{\text{fin}}[p]$   $\triangleright$  saliendo en  $t = 0$  se llega a tiempo
2: para cada tipo  $t$  y paciente alcanzable  $p$  hacer
3:   agregar la ruta singleton  $[0 \rightarrow p \rightarrow 0]$  del tipo  $t$ 
4: fin para
5: para cada tipo de combi  $t$  hacer  $\triangleright$  rutas multi-paciente golosas
6:    $pend \leftarrow$  pacientes alcanzables, ordenados por beneficio decreciente
7:   mientras  $pend \neq \emptyset$  hacer
8:      $ruta \leftarrow []$ ;  $pos \leftarrow$  centro;  $\tau \leftarrow 0$ 
9:     mientras  $|ruta| < \text{asientos}[t]$  hacer
10:       $c \leftarrow$  el paciente de  $pend$  compatible con  $ruta$  y con ventana cumplible llegando en  $\max(\tau +$ 
       $d_{pos,p}, ih_{\text{inicio}}[p])$  que maximiza  $\text{beneficio}[p] - d_{pos,p}$ 
11:      si no existe tal  $c$  entonces salir del ciclo interno
12:      fin si
13:      agregar  $c$  a  $ruta$ ;  $\tau \leftarrow \max(\tau + d_{pos,c}, ih_{\text{inicio}}[c])$ ;  $pos \leftarrow c$ 
14:    fin mientras
15:    si  $ruta = []$  entonces salir del ciclo  $\triangleright$  ningún paciente pudo iniciar ruta
16:    fin si
17:    si  $|ruta| \geq 2$  entonces agregar la columna  $[0 \rightarrow ruta \rightarrow 0]$  del tipo  $t$   $\triangleright$  los singletons ya
    están
18:    fin si
19:    quitar los pacientes de  $ruta$  de  $pend$ 
20:  fin mientras
21: fin para
```

4.3.1 Detalles de implementación

- **Pool global y deduplicación.** Las columnas viven en un único pool compartido por todos los nodos del árbol: una ruta generada en una rama puede reutilizarse en otra. Cada columna se identifica con la clave (tipo de combi, camino) para no insertar duplicados, y en cada nodo el maestro solo ve las columnas *compatibles* con sus decisiones de ramificación.
- **Eliminación de columnas ($K = 25$).** Cada columna lleva un contador de resoluciones consecutivas del maestro en las que valió $y_r = 0$; al llegar a K se marca como eliminada y deja de entrar en los maestros siguientes. Dos salvaguardas mantienen la validez de las cotas: las columnas de la mejor solución entera conocida están *protegidas* (nunca se eliminan) y, si el pricing vuelve a generar una columna eliminada, ésta se *reactiva* en lugar de descartarse, de modo que nunca se declara la convergencia de un LP con columnas atractivas afuera. Se eligió $K = 25$, más conservador que el valor 5 sugerido en el enunciado: mantener columnas de más solo agranda un LP que ya es chico, mientras que eliminar de más obliga al pricing (un MILP costoso) a regenerarlas.
- **Expiración segura del umbral.** Se fija un instante límite global al inicio, y toda llamada a SCIP (maestros y pricings) recibe `limits/time` acotado por el tiempo restante; el pricing se limita además a 10 segundos por llamada. Antes de explorar se reserva una ventana final de $\min(10, \max(3, 0.15 \cdot \text{umbral}))$ segundos para el maestro entero de refuerzo. Si el tiempo se agota en medio de un nodo, se devuelve el último LP resuelto (marcado como no convergido) y se reporta el incumbente vigente.
- **Prioridad de una solución factible.** La combinación de warm-start y reserva final garantiza que la función siempre escriba una solución factible antes de agotar el umbral: aun con umbrales

muy cortos existe al menos el incumbente del warm-start y, en el peor caso, la solución trivial “no operar” ($Z = 0$).

- **Obtención de duales en SCIP.** En el maestro LP se desactivan presolve, heurísticas y propagación: si SCIP transforma o reduce el problema, los duales de las restricciones originales se pierden o quedan corruptos. Además, en problemas de maximización SCIP expone los duales de su problema interno de minimización, por lo que deben *negarse* para recuperar los duales verdaderos. Los duales de las cotas superior e inferior de flota de un mismo tipo se suman para formar μ_t .
- **Variables artificiales penalizadas.** La penalización se fija en $\rho = \sum_p |\text{beneficio}[p]| + \sum_t \text{costo}[t] + 1000$, estrictamente mayor que cualquier mejora posible del objetivo: una artificial solo toma valor positivo si las decisiones del nodo son insatisfacibles, y en ese caso el nodo se poda. Su presencia mantiene el LP factible y con duales bien definidos durante toda la generación de columnas.
- **Reutilización de los modelos de pricing.** El MILP de pricing de cada tipo se construye una sola vez por nodo; entre iteraciones de CG solo se actualiza su función objetivo con los duales nuevos (vía `freeTransform` de PySCIPOpt), siguiendo la recomendación del enunciado de modificar dinámicamente un modelo ya ejecutado en lugar de reconstruirlo desde cero.
- **Tolerancias y salvaguardas.** Ganancia reducida mínima $\varepsilon = 10^{-4}$, tolerancia de integralidad 10^{-5} y de poda 10^{-6} ; tope de 500 nodos como resguardo ante árboles patológicos. El certificado de optimalidad solo se declara si el árbol se agotó sin alcanzar ninguno de los límites.

5 Evaluación Comparativa

5.1 Configuración de Evaluación

- **Instancias:** 4 casos de prueba en carpeta IN/
- **Timeout global:** 500 segundos por instancia y modelo
- **Métricas capturadas:**
 - Mejor objetivo encontrado (Z)
 - Cota dual (relajación LP)
 - Número de restricciones
 - Número de variables
 - Número de variables en modelo maestro (para CG)
 - Tiempo de ejecución

5.2 Tabla Comparativa

Instancia	Métrica	Estrategia			Óptimo
		Salud	SaludCG	Challenger	
test1	Mejor objetivo	390.0	-	390.0	390.0
	Cota dual	390.0	-	390.0	390.0
	# variables	143	-	10	-
	# restricciones	171	-	5	-
	Tiempo (s)	0.45	-	0.03	-
test2	Mejor objetivo	640.0	-	640.0	640.0
	Cota dual	640.0	-	640.0	640.0
	# variables	244	-	23	-
	# restricciones	284	-	8	-
	Tiempo (s)	0.32	-	0.07	-
test3	Mejor objetivo	1110.0	-	1110.0	1110.0
	Cota dual	1110.0	-	1110.0	1110.0
	# variables	367	-	38	-
	# restricciones	427	-	10	-
	Tiempo (s)	0.28	-	0.22	-
test4	Mejor objetivo	20.0	-	20.0	20.0
	Cota dual	20.0	-	25.0	20.0
	# variables	244	-	20	-
	# restricciones	284	-	6	-
	Tiempo (s)	0.41	-	0.15	-

Table 1: Resultados comparativos de las tres estrategias. En negrita se destaca el mejor valor para cada métrica.

5.3 Abreviaciones

- **Mejor objetivo:** Valor de la función objetivo Z encontrado (beneficio neto)
- **Cota dual:** Valor objetivo de la relajación lineal (proporciona cota inferior)
- **# variables:** Cantidad total de variables en el modelo. Para las estrategias basadas en generación de columnas (SaludCG y Challenger) se reporta la cantidad de columnas (rutas) generadas en el problema maestro

- **# restricciones:** Cantidad total de restricciones en el modelo. Para SaludCG y Challenger se reportan las restricciones del maestro ($|P| + |T|$)
- **Tiempo (s):** Tiempo de ejecución en segundos
- -: No aplicable o no completado aún

6 Conclusiones

[A COMPLETAR]

6.1 Hallazgos Principales

- La Estrategia 1 (Salud MILP compacto) resuelve correctamente todas las instancias a optimalidad
- La Estrategia 3 (SaludChallenger, Branch & Price) alcanza y prueba el óptimo en las cuatro instancias. En `test1–test3` la relajación por rutas ya es entera en el nodo raíz; en `test4` el LP raíz es fraccionario (cota 25.0 contra óptimo 20.0) y el algoritmo ramifica (5 nodos) hasta probar optimalidad
- El maestro de la Estrategia 3 requiere órdenes de magnitud menos variables que el modelo compacto (por ejemplo, 38 columnas contra 367 variables en `test3`), lo que anticipa una mejor escalabilidad en instancias grandes
- Los tiempos de ejecución son muy bajos (¡ 1 segundo) para las instancias de prueba
- La validación sin LP (SaludTest) funciona correctamente, confirmando factibilidad de todas las soluciones

6.2 Comportamientos Sorprendentes

[A COMPLETAR]

6.3 Dificultades y Conocimientos Adquiridos

- **Linearización de restricciones:** El uso de Big-M es necesario pero requiere calibración cuidadosa
- **Reconstrucción de rutas:** Extraer la solución desde variables de decisión requiere cuidado en la topología
- **Validación sin LP:** Implementar validación sin programación lineal es posible y útil para verificar factibilidad de manera independiente